

## String Handling

A **String** variable contains a collection of characters surrounded by double quotes ("" )

### Ways of Creating a Java String

#### Using String Literal

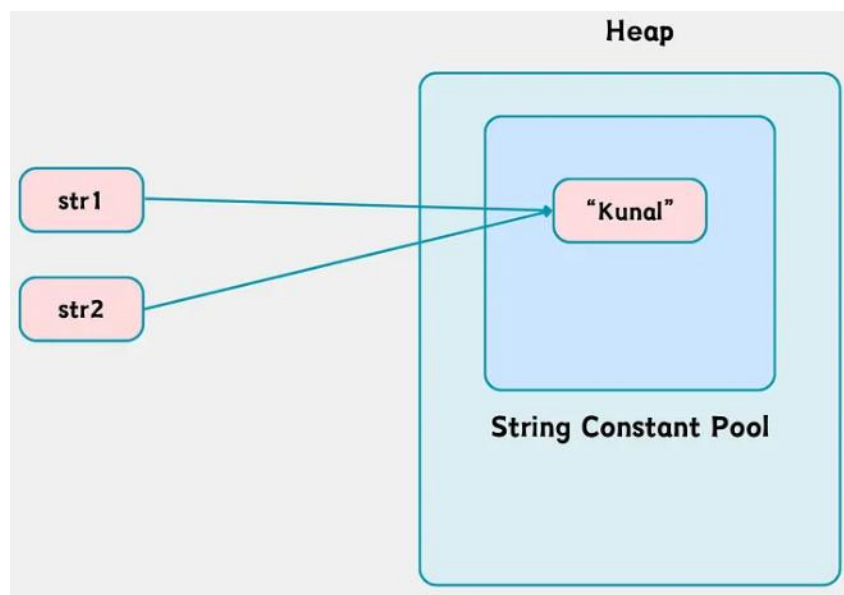
```
String s1 = "Hello";
```

"Hello" is a String literal.

It is stored in the **String Constant Pool**.

If the same literal already exists, Java can reuse the existing object.

```
public class Example1 {  
    public static void main(String[] args) {  
        String s1 = "Kunal";  
        String s2 = "Kunal";  
        if (s1 == s2)  
            System.out.println("Yes");  
        else  
            System.out.println("No");  
    }  
}  
Yes
```



#### Using **new** Keyword

**String s = new String("Hello");**

A new String object is explicitly created.

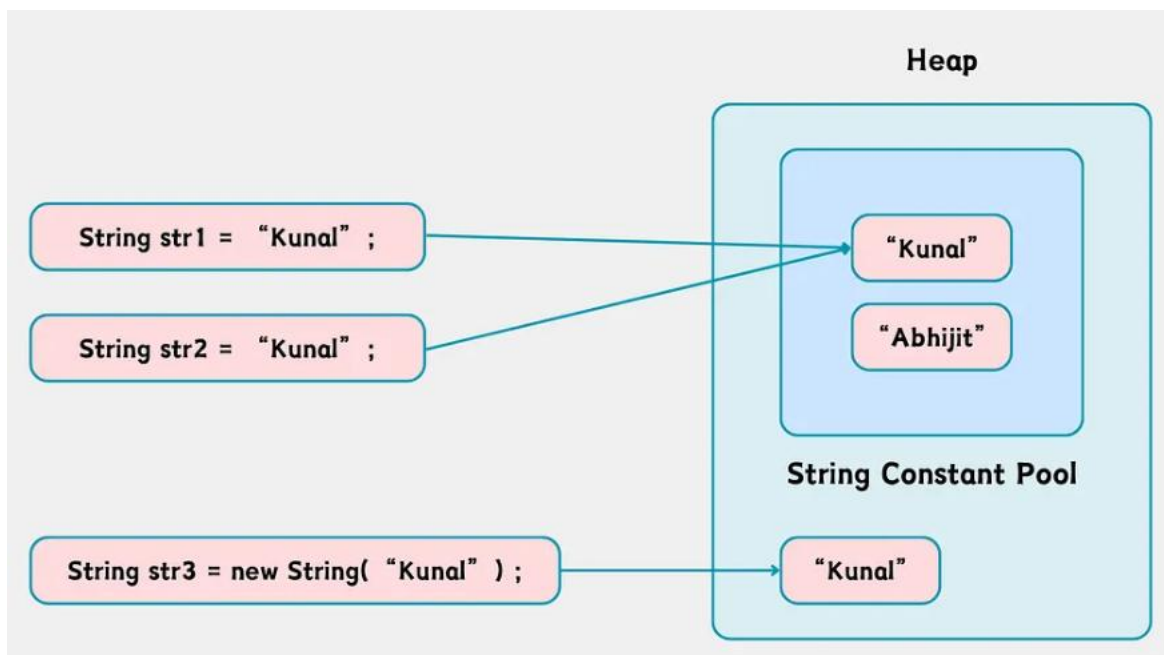
The new keyword creates an object in the **heap**.

```
public class Example2 {  
    public static void main(String[] args) {  
        String s1 = new String("abc");  
        String s2 = new String("abc");  
  
        if (s1 == s2)  
            System.out.println("Yes");  
        else  
            System.out.println("No");  
    }  
}
```

No

Here, s1 and s2 refer to different String objects, even though their contents are the same.

| Method         | Example                         | Object Creation              |
|----------------|---------------------------------|------------------------------|
| String Literal | String s = "Hello";             | Uses String Constant Pool    |
| new keyword    | String s = new String("Hello"); | Creates a new object in Heap |



```
String str1 = new String("Java");
String str2 = "Java";
System.out.println(str1 == str2); // false
```

### Intern()

The `intern()` method forces a string object to be added to the SCP. If the string already exists in the SCP, it returns the existing reference; otherwise, it adds the string to the pool.

|                                                                                                                                                                                        |                                                                                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>String str1 = new String("Hello"); String str2 = str1.intern(); String str3 = "Hello"; System.out.println(str1 == str2); // false System.out.println(str2 == str3); // true</pre> | <pre>String s1 = "TAT"; String s2 = new String("TAT"); String s3 = s2.intern(); System.out.println(s1 == s2); // false System.out.println(s1 == s3); // true System.out.println(s1.equals(s2)) // true</pre> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

`==` Compares references, whereas `equals()` compares String contents.

### String is immutable

Once a String object is created, its content cannot be changed.

String operations create a new String object instead.

```
class Demo {
    public static void main(String[] args) {
        String s = "Java";
        System.out.println("Before: " + s);
        s.concat(" Programming");
        System.out.println("After concat(): " + s);
        s = s.concat(" Programming");
        System.out.println("After assigning: " + s);
    }
}
```

Before: Java

After concat(): Java

After assigning: Java Programming

## StringBuilder

StringBuilder is **mutable**

Mutable → content can be changed.

```
StringBuilder sb = new StringBuilder("Java");
sb.append(" Programming");
System.out.println(sb);
Java Programming
```

## StringTokenizer

StringTokenizer is used to split a string into smaller parts called tokens

```
import java.util.StringTokenizer;
class Demo {
    public static void main(String[] args) {
        String str = "Java is easy";
        StringTokenizer st = new StringTokenizer(str);
        while (st.hasMoreTokens()) {
            System.out.println(st.nextToken());
        }
    }
}
Java
is
easy
```

Here, the **space** is used as the **default delimiter**.

You can also specify your own delimiter:

```
StringTokenizer st = new StringTokenizer("Java,C++,Python", ",");
Java
C++
Python
```

## Common String Methods

| Method                | Description                              | Example                 |
|-----------------------|------------------------------------------|-------------------------|
| length()              | Returns the length of the string         | s.length()              |
| charAt(index)         | Returns the character at the given index | s.charAt(0)             |
| substring(begin, end) | Extracts part of a string                | s.substring(1, 4)       |
| equals()              | Compares string contents                 | s1.equals(s2)           |
| equalsIgnoreCase()    | Compares ignoring case                   | s1.equalsIgnoreCase(s2) |
| compareTo()           | Lexicographical comparison               | s1.compareTo(s2)        |
| concat()              | Concatenates strings                     | s1.concat(s2)           |
| contains()            | Checks if a substring exists             | s.contains("He")        |
| replace()             | Replaces characters or substrings        | s.replace("H", "J")     |
| toUpperCase()         | Converts to uppercase                    | s.toUpperCase()         |
| toLowerCase()         | Converts to lowercase                    | s.toLowerCase()         |
| trim()                | Removes leading and trailing spaces      | s.trim()                |

```
public class Demo {  
    public static void main(String[] args) {  
  
        String s = "Hello Java";  
        String s1 = "Java";  
        String s2 = "java";  
  
        System.out.println("Length: " + s.length());  
  
        System.out.println("Character at index 1: " + s.charAt(1));  
  
        System.out.println("Substring: " + s.substring(0, 5));  
        // Start at index 0 and stop before index 5 i.e. 0, 1, 2, 3, 4  
  
        System.out.println("equals(): " + s1.equals(s2));  
  
        System.out.println("equalsIgnoreCase(): " + s1.equalsIgnoreCase(s2));  
  
        System.out.println("compareTo(): " + s1.compareTo(s2));  
  
        System.out.println("concat(): " + s1.concat(" Programming"));  
  
        System.out.println("contains(): " + s.contains("Java"));  
  
        System.out.println("replace(): " + s.replace("Java", "World"));  
  
        System.out.println("toUpperCase(): " + s.toUpperCase());  
  
        System.out.println("toLowerCase(): " + s.toLowerCase());  
  
        String str = " Hello Java ";  
    }  
}
```

```
System.out.println("Before trim(): [" + str + "]");

System.out.println("After trim(): [" + str.trim() + "]");
}
}
```

Length: 10  
Character at index 1: e  
Substring: Hello  
equals(): false  
equalsIgnoreCase(): true  
compareTo(): -32  
concat(): Java Programming  
contains(): true  
replace(): Hello World  
toUpperCase(): HELLO JAVA  
toLowerCase(): hello java  
Before trim(): [ Hello Java ]  
After trim(): [Hello Java]

```
public class StringMethods {
    public static void main(String[] args) {

        String s = "Java Programming Java";

        // 1. indexOf(int ch)
        System.out.println("1. First 'a': " + s.indexOf('a'));

        // 2. indexOf(int ch, int fromIndex)
        System.out.println("2. 'a' from index 2: " + s.indexOf('a', 2));

        // 3. indexOf(String str)
        System.out.println("3. First \"Java\": " + s.indexOf("Java"));

        // 4. indexOf(String str, int fromIndex)
        System.out.println("4. \"Java\" from index 5: " + s.indexOf("Java", 5));

        // 5. Character not found
        System.out.println("5. 'z' not found: " + s.indexOf('z'));

        // 6. String not found
        System.out.println("6. \"Python\" not found: " + s.indexOf("Python"));

        // 7. startsWith()
        System.out.println("7. Starts with Java: " + s.startsWith("Java"));

        // 8. endsWith()
        System.out.println("8. Ends with Java: " + s.endsWith("Java"));

        // 9. startsWith() - false
```

```
System.out.println("9. Starts with Python: " + s.startsWith("Python"));
```

```
// 10. endsWith() - false
```

```
System.out.println("10. Ends with Python: " + s.endsWith("Python"));
```

```
}  
}
```

1. First 'a': 1
2. 'a' from index 2: 3
3. First "Java": 0
4. "Java" from index 5: 17
5. 'z' not found: -1
6. "Python" not found: -1
7. Starts with Java: true
8. Ends with Java: true
9. Starts with Python: false
10. Ends with Python: false